

Mini RFC: ECN

April 5, 2022

Note: This is a simplified and annotated summary of the QUIC-related IETF RFCs. It's part of a course project at Tsinghua University and not suitable for any other purposes.

We reorder paragraphs in QUIC RFCs and omit many details to improve its readability for beginners.

WARNING: DO NOT distribute this document. Modification of extracts from IETF RFCs is not granted by IETF as original authors retain copyrights of RFCs. This document is for educational purposes only.

1 Overview

Early Congestion Control (ECN) is an enhancement to the IP protocol defined in [RFC 3168](#). It allows network devices to mark two special bits in the IP headers of packets to notify endpoints of congestion in the network. Before ECN, packet drop is the only signal network devices may issue. Network devices may actively limit speeds of certain flows by Random Early Detection (RED, [RFC 7567](#)) to avoid buffer overflow and massive packet drops. ECN serves as a more explicit signal, avoids unnecessary packet drops and provides other benefits as guided in [RFC 8087](#).

Two bits of the ECN field define four possible states for ECN as shown in [Table 1](#).

We briefly introduces ECN here for your convinience. It's very sketchy but enough for this project. Learn more about ECN and active queue management (AQM) from their RFCs if you are interested in this topic.

Bit 0	Bit 1	Name
0	0	Not-ECT
0	1	ECT(1)
1	0	ECT(0)
1	1	CE

Table 1: *The ECN Field in the IP Packet Header*

Endpoints that support ECN sets ECT(1) or ECT(0) in the IP header of packets they send. Network devices supporting ECN recognize this field and modify it to CE if they experience (early) congestion.

QUIC endpoints can use ECN to detect and respond to network congestion. ECN allows an endpoint to set an ECN-Capable Transport (ECT)

codepoint in the ECN field of an IP packet. A network node can then indicate congestion by setting the ECN-CE codepoint in the ECN field instead of dropping the packet [RFC 8087]. Endpoints react to reported congestion by reducing their sending rate in response.

To enable ECN, a sending QUIC endpoint first determines whether a path supports ECN marking and whether the peer reports the ECN values in received IP headers.

2 Generating ECN Signal

2.1 Reporting ECN Counts

The use of ECN requires the receiving endpoint to read the ECN field from an IP packet, which is not possible on all platforms. If an endpoint does not implement ECN support or does not have access to received ECN fields, it does not report ECN counts for packets it receives.

Even if an endpoint does not set an ECT field in packets it sends, the endpoint **MUST** provide feedback about ECN markings it receives, if these are accessible. Failing to report the ECN counts will cause the sender to disable the use of ECN for this connection.

On receiving an IP packet with an ECT(0), ECT(1), or ECN-CE codepoint, an ECN-enabled endpoint accesses the ECN field and increases the corresponding ECT(0), ECT(1), or ECN-CE count. These ECN counts are included in subsequent ACK frames.

Each packet number space maintains separate acknowledgment state and separate ECN counts. Coalesced QUIC packets share the same IP header so the ECN counts are incremented once for each coalesced QUIC packet.

For example, if one each of an Initial, Handshake, and 1-RTT QUIC packet are coalesced into a single UDP datagram, the ECN counts for all three packet number spaces will be incremented by one each, based on the ECN field of the single IP header.

ECN counts are only incremented when QUIC packets from the received IP packet are processed. As such, duplicate QUIC packets are not processed and do not increase ECN counts.

Packets marked with the ECN Congestion Experienced (CE) codepoint in the IP header **SHOULD** be acknowledged immediately, to reduce the peer's response time to congestion events.

2.2 ECN Validation

2.2.1 Packet Drop

It is possible for faulty network devices to corrupt or erroneously drop packets that carry a non-zero ECN codepoint. To ensure connectivity in the presence of such devices, an endpoint validates the ECN counts for each network path and disables the use of ECN on that path if errors are detected.

To perform ECN validation for a new path:

- The endpoint sets an ECT(0) codepoint in the IP header of early outgoing packets sent on a new path to the peer.
- The endpoint monitors whether all packets sent with an ECT codepoint are eventually deemed lost, indicating that ECN validation has failed.

If an endpoint has cause to expect that IP packets with an ECT codepoint might be dropped by a faulty network element, the endpoint could set an ECT codepoint for only the first ten outgoing packets on a path, or for a period of three PTOs. If all packets marked with non-zero ECN codepoints are subsequently lost, it can disable marking on the assumption that the marking caused the loss.

An endpoint thus attempts to use ECN and validates this for each new connection, when switching to a server's preferred address, and on active connection migration to a new path.

Other methods of probing paths for ECN support are possible, as are different marking strategies. Implementations MAY use other methods defined in RFCs; see [RFC8311]. Implementations that use the ECT(1) codepoint need to perform ECN validation using the reported ECT(1) counts.

2.2.2 Incorrect Markings

Erroneous application of ECN-CE markings by the network can result in degraded connection performance. An endpoint that receives an ACK frame with ECN counts therefore validates the counts before using them. It performs this validation by comparing newly received counts against those from the last successfully processed ACK frame. Any increase in the ECN counts is validated based on the ECN markings that were applied to packets that are newly acknowledged in the ACK frame.

If an ACK frame newly acknowledges a packet that the endpoint sent with either the ECT(0) or ECT(1) codepoint set, ECN validation fails if the corresponding ECN counts are not present in the ACK frame. This check detects a network element that zeroes the ECN field or a peer that does not report ECN markings.

ECN validation also fails if the sum of the increase in ECT(0) and ECN-CE counts is less than the number of newly acknowledged packets that were originally sent with an ECT(0) marking. Similarly, ECN validation fails if the sum of the increases to ECT(1) and ECN-CE counts is less than the number of newly acknowledged packets sent with an ECT(1) marking. These checks can detect remarking of ECN-CE markings by the network.

An endpoint could miss acknowledgments for a packet when ACK frames are lost. It is therefore possible for the total increase in ECT(0), ECT(1), and ECN-CE counts to be greater than the number of packets that are newly acknowledged by an ACK frame. This is why ECN counts are permitted to be larger than the total number of packets that are acknowledged.

It's a little sad that ECN should be taken very carefully. One principle of the Internet is, however, *Be strict when sending and tolerant when receiving*.

Validating ECN counts from reordered ACK frames can result in failure. An endpoint **MUST NOT** fail ECN validation as a result of processing an ACK frame that does not increase the largest acknowledged packet number.

ECN validation can fail if the received total count for either ECT(0) or ECT(1) exceeds the total number of packets sent with each corresponding ECT codepoint. In particular, validation will fail when an endpoint receives a non-zero ECN count corresponding to an ECT codepoint that it never applied. This check detects when packets are remarked to ECT(0) or ECT(1) in the network.

If validation fails, then the endpoint **MUST** disable ECN. It stops setting the ECT codepoint in IP packets that it sends, assuming that either the network path or the peer does not support ECN.

Even if validation fails, an endpoint **MAY** revalidate ECN for the same path at any later time in the connection. An endpoint could continue to periodically attempt validation.

Upon successful validation, an endpoint **MAY** continue to set an ECT codepoint in subsequent packets it sends, with the expectation that the path is ECN capable. Network routing and path elements can change mid-connection; an endpoint **MUST** disable ECN if validation later fails.

3 Usage of ECN Signal

Increases of the ECN-CT count in received ACK frames are equivalent to packet drops in the context of congestion control. As shown in [Figure 1](#), endpoints treat ECN-CE increase as an equal signal to packet loss.

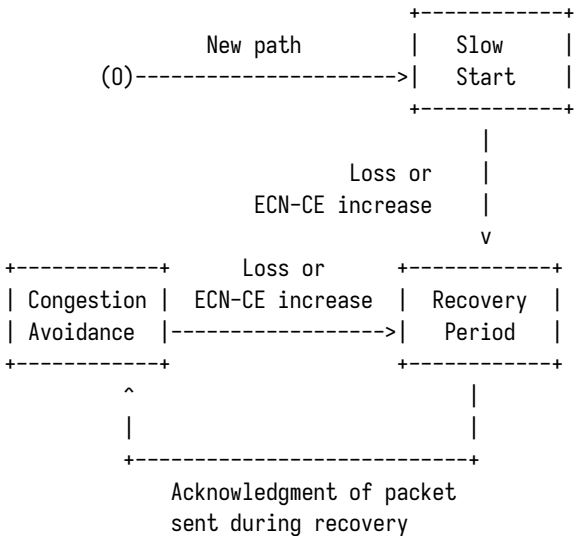


Figure 1: Congestion Control States and Transitions

More specifically, the sender

We have discussed the NewReno congestion control algorithm in [mini RFC: Congestion Control](#). Now all descriptions like “when a packet is lost” should be “when a packet is lost **or the ECN-CE count reported by its peer increases**”.

- exits slow start and enters a recovery period when a packet is lost or when the ECN-CE count reported by its peer increases.
- enters a recovery period when it detects the loss of a packet or when the ECN-CE count reported by its peer increases. A sender that is already in a recovery period stays in it and does not reenter it.
- exits congestion avoidance and enters a recovery period when a packet is lost or when the ECN-CE count reported by its peer increases.

4 Packets and Frames

4.1 Packets

There is no new type of packets introduced in this RFC.

4.2 Frames

4.2.1 ACK Frames (Type = 0x03)

We have discussed the ACK frame with the frame type 0x02 in *mini RFC: Reliable Transmission*. If the frame type is 0x03, ACK frames also contain the cumulative count of QUIC packets with associated ECN marks received on the connection up until this point. QUIC implementations MUST properly handle both types, and, if they have enabled ECN for packets they send, they SHOULD use the information in the ECN section to manage their congestion state.

```
ACK Frame {
    Type (i) = 0x03,
    Largest Acknowledged (i),
    ACK Delay (i),
    ACK Range Count (i),
    First ACK Range (i),
    ACK Range (..) ...,
    ECN Counts (..),
}
```

The ACK frame uses the least significant bit of the type value (that is, type 0x03) to indicate ECN feedback and report receipt of QUIC packets with associated ECN codepoints of ECT(0), ECT(1), or ECN-CE in the packet's IP header. ECN counts are only present when the ACK frame type is 0x03.

When present, there are three ECN counts, as shown in [Figure 2](#).

The ECN count fields are:

- **ECT0 Count:** A variable-length integer representing the total number of packets received with the ECT(0) codepoint in the packet number space of the ACK frame.

```
ECN Counts {  
  ECT0 Count (i),  
  ECT1 Count (i),  
  ECN-CE Count (i),  
}
```

Figure 2: *ECN Count Format*

- **ECT1 Count:** A variable-length integer representing the total number of packets received with the ECT(1) codepoint in the packet number space of the ACK frame.
- **ECN-CE Count:** A variable-length integer representing the total number of packets received with the ECN-CE codepoint in the packet number space of the ACK frame.

ECN counts are maintained separately for each packet number space.